



**S. B. JAIN INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NAGPUR.**

Practical No. 07

Aim: Implement a coding technique to achieve lossless compression and compare the original and compressed file sizes.

Name of Student:

Roll No.:

Semester/Year:V/III

Academic Session: 2026-2027

Date of Performance:

Date of Submission:

Aim: Implement a coding technique to achieve lossless compression and compare the original and compressed file sizes.

OBJECTIVE/EXPECTED LEARNING OUTCOME:

The objectives and expected learning outcome of this practical are:

- To understand and apply lossless compression techniques using suitable coding algorithms such as Huffman coding, Run Length Encoding (RLE), or LZW.
- To implement a coding algorithm in a programming environment (e.g., Python/OpenCV) for compressing digital files without loss of information.
- To compare the file sizes of original and compressed data to evaluate the effectiveness of the chosen lossless compression method.
- To analyze the compression ratio and efficiency achieved by different coding techniques under varying input conditions.
- To enhance problem-solving and programming skills by designing, testing, and validating a lossless compression system.

THEORY:

Image compression is a critical technology in computer vision and Image processing that allows us to store and transmit images more efficiently while maintaining visual quality. Ideally, we'd love to have small files with the best quality. However, we must make the tradeoff and decide which is more important.

This tutorial will teach image compression with OpenCV, covering theory and practical applications. By the end, you'll understand how to compress photos successfully for computer vision projects (or any other projects you might have).

What is Image Compression

Image compression is reducing an image's file size while maintaining an acceptable level of visual quality. There are two main types of compression:

1. **Lossless compression:** Preserves all original data, allowing exact image reconstruction.
2. **Lossy compression:** Discards some data to achieve smaller file sizes, potentially reducing image quality.

Why Compress Images?

If "disk space is cheap," as we often hear, then why compress images at all? At a small scale, image compression doesn't matter much, but at a large scale, it's crucial.

For instance, if you have a few images on your hard drive, you can compress them and

save a few megabytes of data. This is not much of an impact when hard drives are measured in Terabytes. But what if you had 100,000 images on your hard drive? Some basic compression saves real time and money. From a performance perspective, it's the same. If you have a website with a lot of images and 10,000 people visit your website a day, compression matters.

Here's why we do it:

- **Reduced storage requirements:** Store more images in the same space
- **Faster transmission:** Ideal for web applications and bandwidth-constrained scenarios
- **Improved processing speed:** Smaller images are quicker to load and process

Theory Behind Image Compression

Image compression techniques exploit two types of redundancies:

1. **Spatial redundancy:** Correlation between neighboring pixels
2. **Color redundancy:** Similarity of color values in adjacent regions

Spatial redundancy takes advantage of the fact that neighboring pixels tend to have similar values in most natural images. This creates smooth transitions. Many photos “look real” because there is a natural flow from one area to the other. When the neighboring pixels have wildly different values, you get “noisy” images. Pixels changed to make those transitions less “smooth” by grouping pixels into a single color, making the image smaller.

Color redundancy, on the other hand, focuses on how adjacent areas in an image often share similar colors. Think of a blue sky or a green field—large portions of the image might have very similar color values. They can also be grouped together and made into a single color to save space.

OpenCV offers solid tools for working with these ideas. Using spatial redundancy, OpenCV's **cv2.inpaint()** function, for example, fills in missing or damaged areas of a picture using information from nearby pixels. OpenCV lets developers use **cv2.cvtColor()** to translate images between several color spaces regarding color redundancy. This can be somewhat helpful as a preprocessing step in many compression techniques since some color spaces are more effective than others in encoding particular kinds of images.

MODEL:

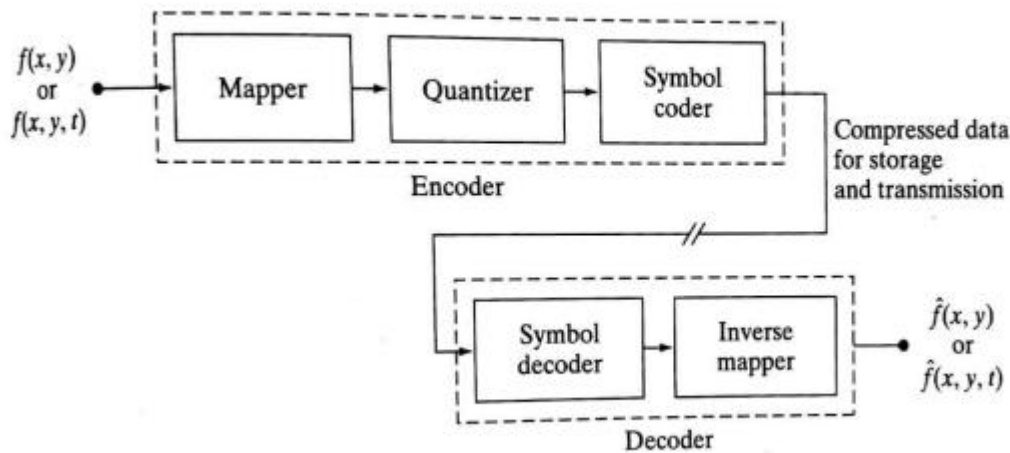


FIGURE 8.5
Functional block diagram of a general image compression system.

- **Mapper** - transforms the image to a (non-visual) format designed to reduce interpixel redundancies. The operation generally is reversible and may or may not reduce directly the amount of data required to represent the image. In video applications, the mapper uses previous video frames to facilitate the removal of temporal redundancy.
- **Quantizer** - reduces psychovisual redundancies by quantizing data deemed less important for visual interpretation (omitted for lossless compression)
- **Symbol encoder** - codes the data efficiently (typically using some form of variable-length coding scheme) and aims to reduce coding redundancies. It generates a fixed or variable length code to represent the quantizer output and maps the output in accordance with the code. In many cases, a variable –length code is used. The shortest code words are assigned to the most frequently occurring quantizer output values- thus minimizing coding redundancy.

MAIN CODE:

```
import cv2
import os
def compression_ratio(original_file, compressed_file):
    original_size = os.path.getsize(original_file)
    compressed_size = os.path.getsize(compressed_file)
    ratio = original_size / compressed_size if compressed_size != 0 else 0
    return ratio, original_size / 1024, compressed_size / 1024 # sizes in KB
```

Image Processing Lab (N-PECCS502P)

- This function computes the compression ratio and returns sizes in KB.
- `os.path.getsize()` returns file sizes in **bytes**.
- `ratio = original_size / compressed_size if compressed_size != 0 else 0` prevents division by zero.
- The function returns `(ratio, original_kb, compressed_kb)` where `original_kb = original_size / 1024`.

```
image_path = 'input_image.png/jpg'
image = cv2.imread(image_path)
if image is None:
    raise FileNotFoundError("Image not found. Check the path.")
jpeg_quality = 30 # lower = higher compression
lossy_output = 'compressed_lossy.jpg'
cv2.imwrite(lossy_output, image, [cv2.IMWRITE_JPEG_QUALITY,
jpeg_quality])
```

- `jpeg_quality = 30` — OpenCV accepts JPEG quality in the range **0–100**.
 - **Lower values** → **higher compression** and **lower visual quality**; **higher values** → better quality and larger file.
- `cv2.imwrite(lossy_output, image, [cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])` writes a JPEG with the requested quality.

```
png_compression = 9
lossless_output = 'compressed_lossless.png'
cv2.imwrite(lossless_output, image, [cv2.IMWRITE_PNG_COMPRESSION,
png_compression])
```

- `png_compression = 9` — PNG compression level range **0–9**; **9** is maximum compression (smaller file but slower).
- `cv2.imwrite(lossless_output, image, [cv2.IMWRITE_PNG_COMPRESSION, png_compression])` writes a PNG (lossless).

```
lossy_ratio, original_kb, lossy_kb = compression_ratio(image_path, lossy_output)
lossless_ratio, _, lossless_kb = compression_ratio(image_path, lossless_output)
```

Image Processing Lab (N-PECCS502P)

- `lossy_ratio, original_kb, lossy_kb = compression_ratio(image_path, lossy_output)`
and similarly for lossless.
- These variables hold the numeric ratio and sizes (KB). The printed ratio is `Original / Compressed`.

```
print(f"Original Size: {original_kb:.2f} KB")
print(f"Lossy JPEG Size: {lossy_kb:.2f} KB (Quality={jpeg_quality})")
print(f"Lossless PNG Size: {lossless_kb:.2f} KB
(Compression={png_compression})")
print(f"Lossy Compression Ratio: {lossy_ratio:.2f}:1")
print(f"Lossless Compression Ratio: {lossless_ratio:.2f}:1")
```

```
cv2.imshow("Original", image)
cv2.imshow("Lossy JPEG", cv2.imread(lossy_output))
cv2.imshow("Lossless PNG", cv2.imread(lossless_output))
cv2.waitKey(0)
cv2.destroyAllWindows()
```

INPUT:

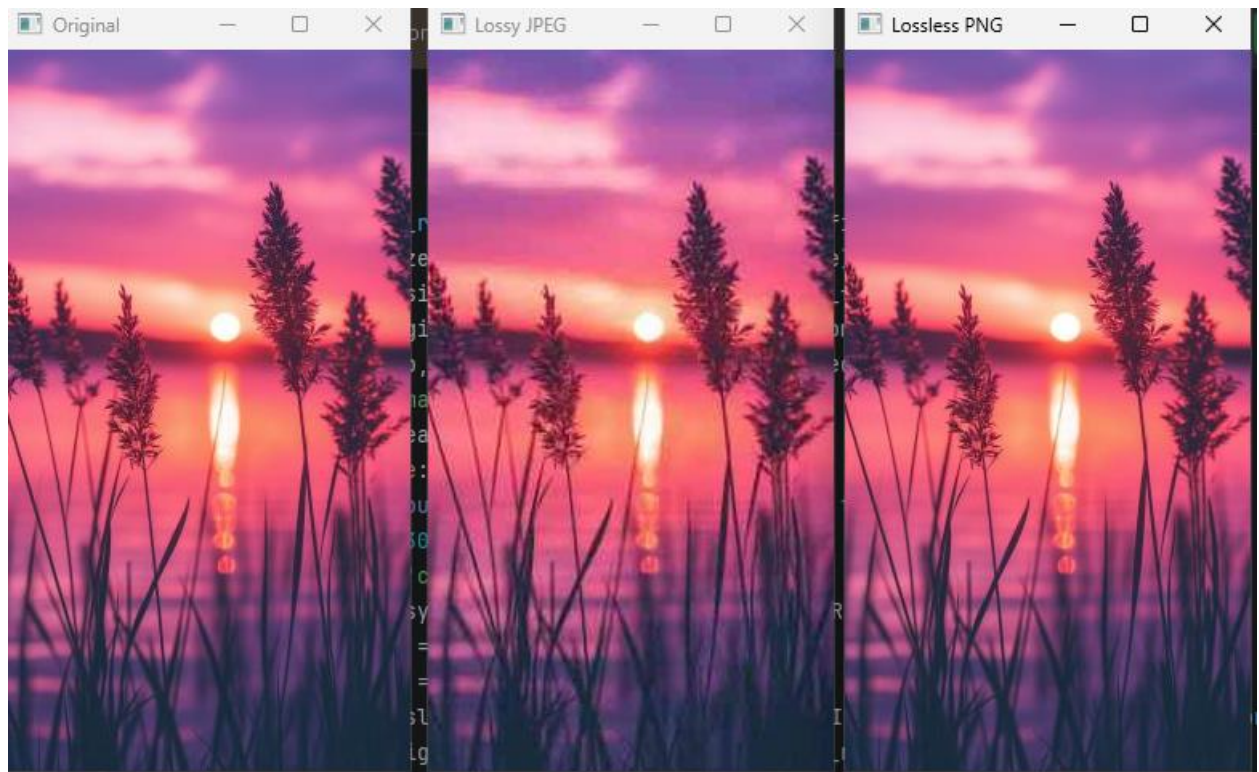


OUTPUT:

Image Processing Lab (N-PECCS502P)

```
main.py x
1 import cv2
2 import os
3 def compression_ratio(original_file, compressed_file): 2 usages
4     original_size = os.path.getsize(original_file)
5     compressed_size = os.path.getsize(compressed_file)
6     ratio = original_size / compressed_size if compressed_size != 0 else 0
7     return ratio, original_size / 1024, compressed_size / 1024
8 image_path = 'image 2.jpg'
9 image = cv2.imread(image_path)
10 if image is None:
11     raise FileNotFoundError("Image not found. Check the path.")
12 jpeg_quality = 30
13 lossy_output = 'compressed_lossy.jpg'
14 cv2.imwrite(lossy_output, image, params=[cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])
15 png_compression = 9
16 lossless_output = 'compressed_lossless.png'
17 cv2.imwrite(lossless_output, image, params=[cv2.IMWRITE_PNG_COMPRESSION, png_compression])
18 lossy_ratio, original_kb, lossy_kb = compression_ratio(image_path, lossy_output)
19 lossless_ratio, _, lossless_kb = compression_ratio(image_path, lossless_output)
20 print(f"Original Size: {original_kb:.2f} KB")
21 print(f"Lossy JPEG Size: {lossy_kb:.2f} KB (Quality={jpeg_quality})")
22 print(f"Lossless PNG Size: {lossless_kb:.2f} KB (Compression={png_compression})")
23 print(f"Lossy Compression Ratio: {lossy_ratio:.2f}:1")
24 print(f"Lossless Compression Ratio: {lossless_ratio:.2f}:1")
25 cv2.imshow(winname="Original", image)
26 cv2.imshow(winname="Lossy JPEG", cv2.imread(lossy_output))
27 cv2.imshow(winname="Lossless PNG", cv2.imread(lossless_output))
28 cv2.waitKey(0)
29 cv2.destroyAllWindows()
```

```
main x
Original Size: 19.03 KB
Lossy JPEG Size: 8.66 KB (Quality=30)
Lossless PNG Size: 156.72 KB (Compression=9)
Lossy Compression Ratio: 2.20:1
Lossless Compression Ratio: 0.12:1
```



Extra Code:

For RLC, LZW

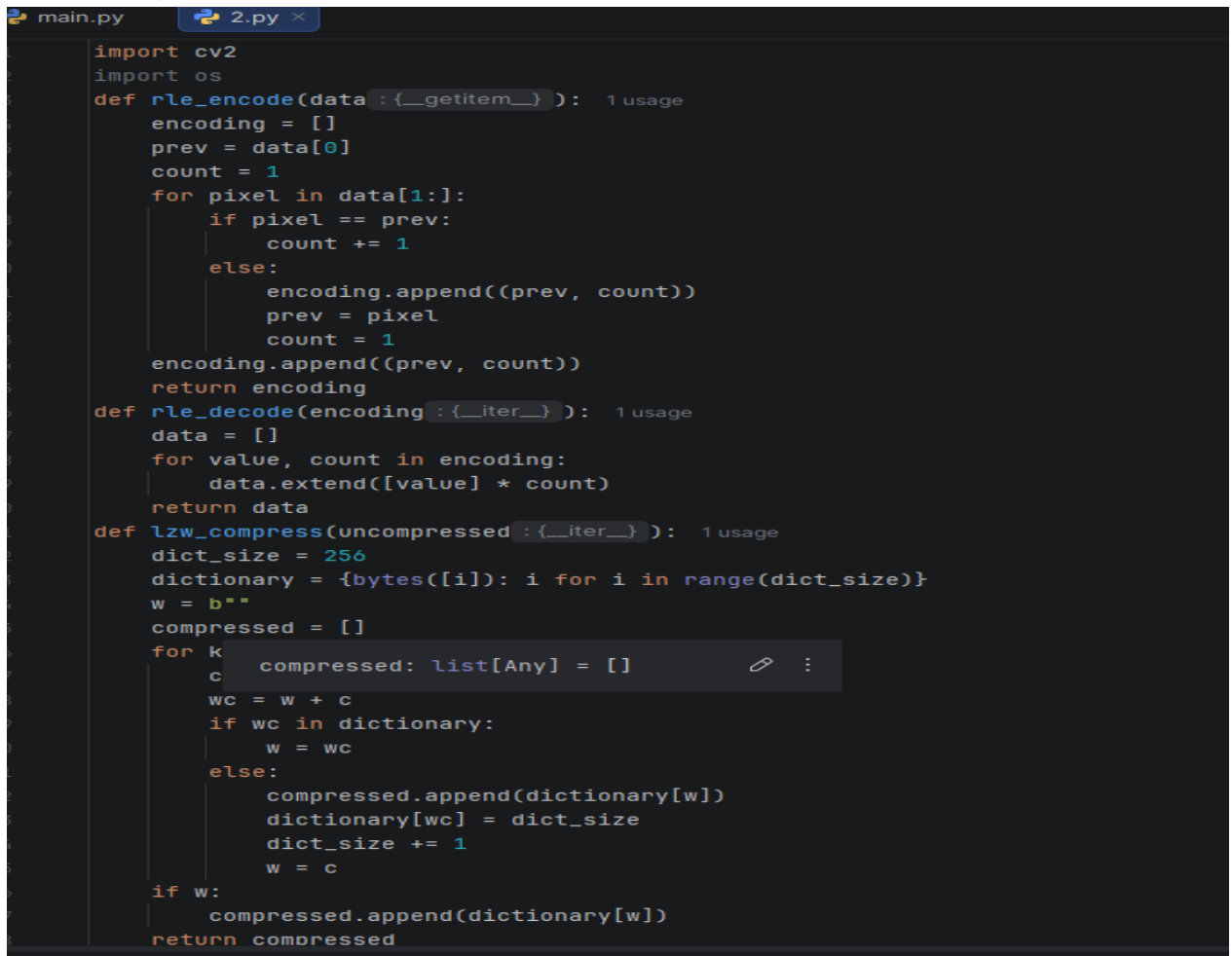
```
import cv2
import os
from collections import defaultdict
def rle_encode(data):
    """Run Length Encode a 1D list of values."""
    encoding = []
    prev = data[0]
    count = 1
    for pixel in data[1:]:
        if pixel == prev:
            count += 1
        else:
            encoding.append((prev, count))
            prev = pixel
            count = 1
    encoding.append((prev, count))
    return encoding
def rle_decode(encoding):
    """Run Length Decode back to original data."""
    data = []
    for value, count in encoding:
        data.extend([value] * count)
    return data
def lzw_compress(uncompressed):
    dict_size = 256
    dictionary = {bytes([i]): i for i in range(dict_size)}
    w = b""
    compressed = []
    for k in uncompressed:
        c = bytes([k])
        wc = w + c
        if wc in dictionary:
            w = wc
```



```
    else:
        compressed.append(dictionary[w])
        dictionary[wc] = dict_size
        dict_size += 1
        w = c
    if w:
        compressed.append(dictionary[w])
    return compressed
def lzw_decompress(compressed):
    dict_size = 256
    dictionary = {i: bytes([i]) for i in range(dict_size)}
    w = bytes([compressed.pop(0)])
    result = bytearray(w)
    for k in compressed:
        if k in dictionary:
            entry = dictionary[k]
        elif k == dict_size:
            entry = w + w[:1]
        else:
            raise ValueError("Bad compressed k: %s" % k)
        result += entry

        dictionary[dict_size] = w + entry[:1]
        dict_size += 1
        w = entry
    return list(result)
image_path = 'input_image.jpg' # Replace with your image
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Image not found.")
pixels = img.flatten().tolist()
rle_encoded = rle_encode(pixels)
rle_size_bytes = len(rle_encoded) *
rle_decoded = rle_decode(rle_encoded)
assert rle_decoded == pixels, "RLE decompression failed!"
```

```
lzw_encoded = lzw_compress(pixels)
lzw_size_bytes = len(lzw_encoded) * 2 # approximate each code as 2 bytes
lzw_decoded = lzw_decompress(lzw_encoded.copy())
assert lzw_decoded == pixels, "LZW decompression failed!"
original_size_bytes = len(pixels) # one byte per pixel
rle_ratio = original_size_bytes / rle_size_bytes if rle_size_bytes else 0
lzw_ratio = original_size_bytes / lzw_size_bytes if lzw_size_bytes else 0
print("Original Image Size (approx):", original_size_bytes, "bytes")
print("RLE Compressed Size (approx):", rle_size_bytes, "bytes")
print("LZW Compressed Size (approx):", lzw_size_bytes, "bytes")
print(f"RLE Compression Ratio: {rle_ratio:.2f}:1")
print(f"LZW Compression Ratio: {lzw_ratio:.2f}:1")
cv2.imshow("Original Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

A screenshot of a Python IDE with two tabs: 'main.py' and '2.py'. The '2.py' tab is active and contains the following code:

```
import cv2
import os
def rle_encode(data : (__getitem__)): 1 usage
    encoding = []
    prev = data[0]
    count = 1
    for pixel in data[1:]:
        if pixel == prev:
            count += 1
        else:
            encoding.append((prev, count))
            prev = pixel
            count = 1
    encoding.append((prev, count))
    return encoding
def rle_decode(encoding : (__iter__)): 1 usage
    data = []
    for value, count in encoding:
        data.extend([value] * count)
    return data
def lzw_compress(uncompressed : (__iter__)): 1 usage
    dict_size = 256
    dictionary = {bytes([i]): i for i in range(dict_size)}
    w = b""
    compressed = []
    for k, c:
        compressed: list[Any] = []
        wc = w + c
        if wc in dictionary:
            w = wc
        else:
            compressed.append(dictionary[w])
            dictionary[wc] = dict_size
            dict_size += 1
            w = c
    if w:
        compressed.append(dictionary[w])
    return compressed
```

Image Processing Lab (N-PECCS502P)

```
main.py 2.py x
1 def lzw_compress(uncompressed : (iter_)): 1 usage
2     return compressed
3
4 def lzw_decompress(compressed : (pop, _iter_)): 1 usage
5     dict_size = 256
6     dictionary = {i: bytes([i]) for i in range(dict_size)}
7     w = bytes([compressed.pop(0)])
8     result = bytearray(w)
9     for k in compressed:
10         if k in dictionary:
11             entry = dictionary[k]
12         elif k == dict_size:
13             entry = w + w[1:]
14         else:
15             raise ValueError("Bad compressed k: %s" % k)
16         result += entry
17
18         dictionary[dict_size] = w + entry[1:]
19         dict_size += 1
20         w = entry
21     return list(int)(result)
22
23 image_path = 'image 2.jpg'
24 img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
25
26 if img is None:
27     raise FileNotFoundError("Image not found.")
28 pixels = img.flatten().tolist()
29 rle_encoded = rle_encode(pixels)
30 rle_size_bytes = len(rle_encoded) * 2 # approximate: 1 byte value + 1 byte count
31 rle_decoded = rle_decode(rle_encoded)
32 assert rle_decoded == pixels, "RLE decompression failed!"
33 lzw_encoded = lzw_compress(pixels)
34 lzw_size_bytes = len(lzw_encoded) * 2 # approximate each code as 2 bytes
35 lzw_decoded = lzw_decompress(lzw_encoded.copy())
36 assert lzw_decoded == pixels, "LZW decompression failed!"
37 original_size_bytes = len(pixels) # one byte per pixel
38 rle_ratio = original_size_bytes / rle_size_bytes if rle_size_bytes else 0
39 lzw_ratio = original_size_bytes / lzw_size_bytes if lzw_size_bytes else 0
```

Type your text

```
CS24035 01
> .venv
2.py
compress
compress
image 1.jp
image 2.jp
image 3.jp
main.py
External Libr
Scratches an
01 if img is None:
02     raise FileNotFoundError("Image not found.")
03
04 pixels = img.flatten().tolist()
05 rle_encoded = rle_encode(pixels)
06 rle_size_bytes = len(rle_encoded) * 2 # approximate: 1 byte value + 1 byte count
07 rle_decoded = rle_decode(rle_encoded)
08 assert rle_decoded == pixels, "RLE decompression failed!"
09 lzw_encoded = lzw_compress(pixels)
10 lzw_size_bytes = len(lzw_encoded) * 2 # approximate each code as 2 bytes
11 lzw_decoded = lzw_decompress(lzw_encoded.copy())
12 assert lzw_decoded == pixels, "LZW decompression failed!"
13 original_size_bytes = len(pixels) # one byte per pixel
14 rle_ratio = original_size_bytes / rle_size_bytes if rle_size_bytes else 0
15 lzw_ratio = original_size_bytes / lzw_size_bytes if lzw_size_bytes else 0
16 print("Original Image Size (approx):", original_size_bytes, "bytes")
17 print("RLE Compressed Size (approx):", rle_size_bytes, "bytes")
18 print("LZW Compressed Size (approx):", lzw_size_bytes, "bytes")
19 print(f"RLE Compression Ratio: {rle_ratio:.2f}:1")
20 print(f"LZW Compression Ratio: {lzw_ratio:.2f}:1")
21 cv2.imshow(winname="Original Image", img)
22 cv2.waitKey(0)
23 cv2.destroyAllWindows()
24
main 2 x
C:\Users\STUDENT\Desktop\CS24035\.venv\Scripts\python.exe C:\Users\STUDENT\Desktop\
Could not find platform independent libraries <prefix>
Original Image Size (approx): 113850 bytes
RLE Compressed Size (approx): 166876 bytes
LZW Compressed Size (approx): 94140 bytes
RLE Compression Ratio: 0.68:1
LZW Compression Ratio: 1.21:1
```



Type your text

CONCLUSION:

DISCUSSION QUESTIONS:

1. What do we mean by lossless compression?

2. Name any two coding techniques used for lossless compression.

3. How do we find the file size of an image in a program?

4. What is the compression ratio, and how is it calculated?

5. Why do we compare original and compressed file sizes after compression?

REFERENCES:

- <https://www.adobe.com/uk/creativecloud/photography/discover/lossy-vs-lossless.html>
- <https://www.opencvhelp.org/tutorials/advanced/image-compression/>
- <https://www.cloudflare.com/en-gb/learning/performance/glossary/what-is-image-compression/>
- <https://www.geeksforgeeks.org/machine-learning/what-is-image-compression/>
- <https://cloudinary.com/glossary/image-compression-algorithms>
- <https://developers.google.com/speed/webp/docs/compression>